

Atelier — Une commande contient des billets

À ouvrir après 1-COURS.pdf du dossier projets/05-commande/.

1–3 périodes. Deux classes. Un total calculé par les objets.

Le dossier `depart/` de cet exercice contient les fichiers de départ. Vous n'avez pas à ouvrir un terminal.

Où l'on va

Au guichet, un billet connaît son prix et ses places restantes. Une commande, elle, rassemble plusieurs billets. Dans cet atelier, vous allez construire d'abord un billet seul, puis une commande qui contient des billets et leur demande leur prix final.

À la fin, votre écran ressemble à ceci. Les trois lignes et le total viennent des objets ; aucun montant n'est écrit dans le HTML.

CAISSE DE LA SALLE

Les lignes Le total

SAS · COMMANDE · DEUX CLASSES

Trois billets, une commande, un total.

Commande tient un tableau de Billet. Le total n'est pas tapé dans le HTML : il vient de total().

Les lignes

Paléo, grande scène — samedi PLACES RESTANTES 0 PRIX FINAL 40,50 francs COMPLET	Club de la Gare — jazz PLACES RESTANTES 2 PRIX FINAL 19,80 francs OUVERT	Salle des fêtes — chorale PLACES RESTANTES 80 PRIX FINAL 12,00 francs OUVERT
---	--	--

Le total
Total : 72,30 francs
 Commande.total() additionne les prixFinal() des trois billets. Paléo (0 place) et le jazz (2 places) ont le tarif réduit : places < 3.

Corrigé : Billet + Commande · pas de base.

Sur un téléphone de 390 pixels de large, les lignes s'empilent et le total reste lisible.



Vous créez exactement ceci

```
sas/  
├─ commande/  
│  └─ index.html  
│  └─ css/  
│      └─ style.css  
├─ js/  
└─ commande.js
```

Le départ contient déjà le HTML et le CSS. Votre travail se fait dans `js/commande.js`. Les deux classes `Billet` et `Commande` vivent dans ce même fichier.

Étape 1 — Ouvrir le départ

Ouvrez `sas/commande/index.html` par un double-clic. Vous devez voir le titre, une zone « Les lignes » vide et une zone « Le total » vide. Le vide est normal : aucun objet n'a encore été créé.

- [] L'arbre correspond exactement au dessin ci-dessus.
- [] Le titre « Trois billets, une commande, un total » est visible.
- [] Aucune carte de billet n'est encore affichée.

Si la page n'est pas mise en forme, vérifiez que `index.html` charge `css/style.css`. Vous n'avez pas à recopier la page ni son style depuis ce PDF.

Étape 2 — Fabriquer un seul billet

Vous avez déjà rencontré des données qui vont ensemble : un titre, un prix et un nombre de places. Une **classe** sert de moule. `new Billet(...)` produit un billet réel à partir de ce moule.

Ouvrez `js/commande.js`. Le constructeur est prêt. Les méthodes sont présentes, mais leurs règles restent à écrire.

Dans `estComplet()`, remplacez le commentaire par la règle nommée **billet complet** : elle est vraie quand `places === 0`.

Dans `prixFinal()`, écrivez la règle nommée **tarif de dernière minute** : quand `places < 3`, retourner `prix × 0,9` ; sinon retourner `prix`. Un billet complet bénéficie aussi de cette règle, car zéro est inférieur à trois.

Sous les deux classes, fabriquez seulement le billet Paléo :

```
const paleo = new Billet("Paléo, grande scène – samedi", 45, 0);
```

Ouvrez les outils de développement du navigateur, puis la console. Tapez successivement :

```
paleo.titre;  
paleo.estComplet();  
paleo.prixFinal();
```

Vous devez lire le titre, puis `true`, puis `40.5`. Vous venez de parler à un objet précis : vous lui avez demandé son état et son prix final.

Étape 3 — Créer une commande vide

Un billet connaît son propre prix. Il ne connaît pas les autres billets achetés avec lui. La classe `Commande` représente le contenant : chaque nouvel objet `Commande` commence avec un tableau vide.

Dans le constructeur de `Commande`, vérifiez que `this.billets` reçoit un tableau vide. Complétez ensuite `ajouter(billet)` avec le geste **ajouter le billet reçu au tableau `this.billets`**.

Sous `const paleo = ...`, créez une commande, puis ajoutez-y ce billet :

```
const commande = new Commande();
```

```
commande.ajouter(paleo);
```

Dans la console, tapez `commande.billets.length`. Vous devez obtenir `1`. `Commande` n'est pas un billet : elle **contient** un billet.

Étape 4 — La commande demande les prix

Le prix final dépend des places restantes. Si `total()` relit `prix` et refait la réduction, la même règle se retrouve à deux endroits. La commande doit donc demander `prixFinal()` à chaque billet.

Complétez `total()` dans la classe `Commande` :

- 1 commencer avec une variable `somme` égale à `0` ;
- 2 parcourir `this.billets` ;
- 3 à chaque passage, ajouter le résultat de `billet.prixFinal()` ;
- 4 retourner `somme` après la boucle.

La ligne importante est un appel de méthode, pas une lecture de `billet.prix`. Dans la console, tapez `commande.total()`. Avec le seul billet `Paléo`, vous devez obtenir `40.5`.

Étape 5 — Trois billets, puis l'affichage

Le premier objet répond correctement. Vous pouvez maintenant ajouter les deux autres billets de la commande actuelle.

Titre	prix	places	prix final attendu
Paléo, grande scène — samedi	45	0	40,50
Club de la Gare — jazz	22	2	19,80
Salle des fêtes — chorale	12	80	12,00

Gardez `paleo`, puis ajoutez deux nouveaux objets avec `commande.ajouter(new Billet(...))`. Dans la console, `commande.total()` doit maintenant répondre environ `72.3`.

Le bas du fichier de départ contient une zone `AFFICHAGE`. Complétez-la pour parcourir `commande.billets`. Pour chaque billet, créez un `article`, puis utilisez :

- `billet.titre` pour le titre ;
- `billet.prixFinal()` pour le prix affiché ;
- `billet.estComplet()` pour choisir « Complet » ou « Ouvert ».

Voici le gabarit d'affichage. Il ne contient aucune règle de prix : il demande les réponses aux objets.

```
const liste = document.getElementById("liste");

commande.billets.forEach((billet) => {
  const article = document.createElement("article");
  article.className = "fiche";
  const prix = billet.prixFinal().toFixed(2).replace(".", ",");
  article.innerHTML = `
<h3>${billet.titre}</h3>
<p>${billet.places} places · ${prix} francs</p>
<p class="badge">${billet.estComplet() ? "Complet" : "Ouvert"}</p>
`;
  liste.appendChild(article);
});
```

Enfin, demandez `commande.total()` pour remplir l'élément `#somme`. La page affiche les réponses des objets ; elle ne recalcule ni la réduction ni le total.

```
const somme = commande.total().toFixed(2).replace(".", ",");
document.getElementById("somme").textContent = `Total : ${somme} francs`;
```

Rechargez. Vous devez voir trois lignes et **Total : 72,30 francs**. Si vous lisez 79,00, cherchez une lecture de `.prix` dans `total()` : la commande doit appeler `.prixFinal()`.

Réussi si

- ☐ Vous pouvez montrer le premier objet `paleo` avant les deux autres.
- ☐ Un nouvel objet `Commande` commence avec un tableau vide.
- ☐ `ajouter(billet)` range le billet dans `this.billets`.
- ☐ `total()` demande `prixFinal()` à chaque billet.
- ☐ Les trois lignes affichent Paléo, le Club de la Gare et la chorale.
- ☐ Le total 72,30 n'est écrit ni dans le HTML ni dans le JavaScript.
- ☐ La page reste lisible à 390 pixels de large.

AVEC L'IA

Vous avez écrit les règles et `total()` vous-même. Montrez seulement les deux classes à une IA et demandez : « Expliquez en quatre phrases pourquoi `Commande.total()` appelle `Billet.prixFinal()` au lieu de lire `prix`. » Vérifiez sa réponse dans votre code ; ne lui demandez pas de refaire la billetterie.

POUR LES RAPIDES

Ajoutez un quatrième billet avec `commande.ajouter(...)`. Rechargez : une nouvelle ligne apparaît et le total suit. Puis changez le prix de la chorale dans son `new Billet(...)` et observez de nouveau le total, sans toucher au HTML.

Livrable

Rendez le dossier `sas/commande/` avec ses trois fichiers. La page doit s'ouvrir par double-clic. À l'oral, montrez `total()` et expliquez pourquoi la commande demande son prix final à chaque billet.